Portfolio Analyzer

Descripción técnica y roadmap completo del proyecto

1. Descripción del proyecto

Portfolio Analyzer es una aplicación para analizar carteras de inversión a partir de datos históricos de mercado. El sistema recibe una lista de activos y sus pesos, obtiene y procesa sus precios históricos y genera métricas de rentabilidad, riesgo, diversificación y rendimiento ajustado al riesgo.

El proyecto puede evolucionar desde un análisis realizado en Python hasta una aplicación completa con motor de analytics, API REST, base de datos, dashboard web, optimización de carteras y backtesting.

2. Objetivos funcionales

• Cargar datos históricos de diferentes activos.

• Construir una cartera a partir de activos y pesos.

• Calcular rentabilidad individual y de cartera.

• Medir volatilidad y diferentes métricas de riesgo.

• Analizar correlaciones, covarianzas y beta.

• Calcular Sharpe, Sortino, VaR y Maximum Drawdown.

• Comparar la cartera con un benchmark.

• Visualizar la evolución y el riesgo de la cartera.

• Optimizar los pesos de la cartera.

• Evaluar estrategias mediante backtesting.

3. Ejemplo de entrada

```
Activos:
AAPL → 30%
MSFT → 25%
GOOGL → 20%
SPY → 25%

Periodo: 2020-01-01 → 2026-09-01
Inversión inicial: 10.000 €
Benchmark: S&P 500
```

4. Resultados esperados

Categoría	Métricas / resultados
Rentabilidad	Total return, annualized return, monthly returns, yearly returns, final portfolio value
Riesgo	Volatility, covariance, correlation, beta, VaR
Drawdown	Maximum drawdown, drawdown periods, recovery
Risk-adjusted	Sharpe Ratio, Sortino Ratio
Comparación	Portfolio vs benchmark
Diversificación	Correlation matrix, allocation, risk contribution

Categoría	Métricas / resultados
Optimización	Minimum variance, maximum Sharpe, efficient frontier

5. Roadmap por fases

Fase	Módulo	Tiempo	Contenido
Fase 0	Preparación	2–3 días	Crear repositorio y entorno. Python, Git, NumPy, pandas, matplotlib y SciPy.
Fase 1	Market Data	1 semana	Descarga, almacenamiento y limpieza de precios históricos. Fechas, series temporales y valores ausentes.
Fase 2	Returns	1 semana	Daily returns, cumulative returns, annualized return y análisis mensual/anual.
Fase 3	Portfolio	1 semana	Activos, pesos, validación de pesos, portfolio value y portfolio returns.
Fase 4	Risk	1–2 semanas	Volatilidad, covarianza, correlación y matriz de covarianzas.
Fase 5	Portfolio Risk	1 semana	Volatilidad de cartera, risk contribution y beta.
Fase 6	Risk-adjusted Performance	1 semana	Sharpe, Sortino, Maximum Drawdown e Historical VaR.
Fase 7	Benchmark	3–4 días	Comparar rendimiento y riesgo frente a un benchmark.
Fase 8	Rolling Analytics	1 semana	Rolling volatility, rolling Sharpe, rolling beta y rolling correlation.
Fase 9	Refactor + Testing	2 semanas	Pasar de notebooks a módulos mantenibles. Tests unitarios e integración.
Fase 10	REST API	1–2 semanas	FastAPI y endpoints para portfolios, performance y risk.
Fase 11	Database	1 semana	PostgreSQL para usuarios, portfolios, holdings, assets, prices y resultados.
Fase 12	Dashboard	2 semanas	Interfaz web para consultar carteras, métricas, gráficos y análisis.
Fase 13	Docker + CI/CD	1 semana	Containerización, automatización de tests, build y deployment.
Fase 14	Optimization	2 semanas	Minimum variance, maximum Sharpe y Efficient Frontier.
Fase 15	Backtesting	2–3 semanas	Motor de backtesting con posiciones, P&L;, costes, slippage, turnover y métricas de riesgo.

6. Conceptos matemáticos principales

Daily Return: R■ = P■ / P■■■ – 1

Portfolio Return: R■ = Σ w■R■

Annualized Volatility: σannual = σdaily × √252

Portfolio Volatility: σ■ = √(w■Σw)

Sharpe Ratio: Sharpe = (R■ – Rf) / σ■

Drawdown: DD■ = (V■ – Peak■) / Peak■

7. Estructura del código

```
portfolio-analyzer/
■■■ README.md
■■■ requirements.txt
■■■ .gitignore
■■■ data/
■■■ notebooks/
■ ■■■ 01_data_exploration.ipynb
■ ■■■ 02_returns.ipynb
■ ■■■ 03_risk.ipynb
■ ■■■ 04_optimization.ipynb
■■■ src/
■ ■■■ data/
■ ■ ■■■ loader.py
■ ■ ■■■ cleaning.py
■ ■■■ portfolio/
■ ■ ■■■ portfolio.py
■ ■ ■■■ returns.py
■ ■■■ risk/
■ ■ ■■■ volatility.py
■ ■ ■■■ covariance.py
■ ■ ■■■ drawdown.py
■ ■ ■■■ var.py
■ ■ ■■■ beta.py
■ ■■■ analytics/
■ ■■■ metrics.py
■■■ tests/
```

8. Arquitectura objetivo

```
Frontend / Dashboard
↓
REST API — FastAPI
↓
Portfolio & Analytics Engine
↓
PostgreSQL + Market Data Storage
↓
External Market Data Provider
```

La separación entre adquisición de datos, lógica de cartera, cálculo de métricas, API y persistencia permite que el proyecto sea mantenible y pueda crecer sin convertir todo el código en un único script.

9. Portfolio Optimization

En una fase avanzada, el sistema deja de limitarse a analizar una cartera y puede buscar carteras óptimas. Se pueden implementar restricciones como pesos que sumen 100%, límites máximos por activo, long-only y una rentabilidad objetivo.

Minimum Variance: minimizar w■Σw.
Maximum Sharpe: maximizar (Rp – Rf) / σp.
Efficient Frontier: representar las carteras con mejor combinación posible de rentabilidad y riesgo.

10. Backtesting

El motor de backtesting permite evaluar estrategias sobre datos históricos. Como primera estrategia se puede implementar un cruce de medias móviles: comprar cuando MA20 > MA50 y vender cuando MA20 < MA50.

El motor debe incorporar posiciones, capital, P&L;, transaction costs, slippage, turnover, drawdown, Sharpe y Sortino. La metodología debe controlar especialmente look-ahead bias, survivorship bias y data leakage.

11. Tecnologías

Área	Tecnologías
Lenguaje	Python
Data	pandas, NumPy
Visualización	matplotlib
Cálculo científico	SciPy
Backend	FastAPI
Database	PostgreSQL
Frontend	React u otra solución web
Testing	pytest
Version control	Git + GitHub
Containers	Docker
CI/CD	GitHub Actions

12. Criterio de finalización

El proyecto se considera completo cuando un usuario puede crear una cartera, seleccionar un periodo, obtener datos, calcular métricas de performance y riesgo, comparar contra un benchmark, visualizar resultados y, en la versión avanzada, optimizar la cartera y ejecutar un backtest.

Las funcionalidades avanzadas pueden incorporarse progresivamente. No es necesario construir toda la arquitectura final antes de tener una primera versión funcional.